

SQL i nie tylko

Wykład szósty

dr inż. Marcin Ciura, Uniwersytet Komisji Edukacji Narodowej

Proszę zrobić
notatki :-)

Plan na dziś

Plan na dziś

Z tego wykładu dowiedziecie się:

- + o klawzulach UNION i UNION ALL
- + o klawzuli INTERSECT
- + o klawzuli EXCEPT
- + o wspólnych wyrażeniach tabelowych (Common Table Expressions, CTE), czyli o klawzuli WITH
- + o innych rodzajach zapytań zagnieżdżonych

Przepraszam was za to, że dziś
znów nie będzie anegdoty
Za to za tydzień będą dwie :-)

Klauzule UNION
i UNION ALL

Klauzule UNION i UNION ALL

Klauzula UNION łączy wyniki dwóch zapytań, usuwając duplikowane wiersze

Klauzula UNION ALL łączy wszystkie wiersze z wyników dwóch zapytań, zachowując powtórzenia

Klauzule UNION i UNION ALL

Oba zapytania muszą zwracać tyle samo kolumn

Kolumny na odpowiadających sobie pozycjach
powinny mieć zgodne typy

Nazwy kolumn wyniku pochodzą z pierwszego
zapytania

Klauzule UNION i UNION ALL

Przykład 1: Znaleźć numery tych pracowników, którzy są szefami działów lub kierownikami zespołów

Klauzule UNION i UNION ALL

```
SELECT id_pracownika_szefa_działu AS id_pracownika  
FROM Dział  
UNION  
SELECT id_pracownika_kierownika_zespołu  
FROM Zespół;
```

id_pracownika
1
2
3

Klauzule UNION i UNION ALL

```
SELECT id_pracownika_szefa_działu AS id_pracownika  
FROM Dział  
UNION ALL  
SELECT id_pracownika_kierownika_zespołu  
FROM Zespół;
```

id_pracownika
3
1
2
1
2

Klauzule UNION i UNION ALL

Przykład 2: Znaleźć imiona tych pracowników, którzy są szefami działów lub kierownikami zespołów

Klauzule UNION i UNION ALL

Rozwiązanie 1:

```
SELECT imię  
FROM Pracownik p  
JOIN Dział d ON p.id_pracownika = d.id_pracownika_szefa_działu  
UNION  
SELECT imię  
FROM Pracownik p  
JOIN Zespół z ON id_pracownika = z.id_pracownika_kierownika_zespołu;
```

imię
Kasia
Olga
Wiktor

Klauzule UNION i UNION ALL

Rozwiązanie 2:

```
SELECT imię  
FROM Pracownik JOIN (  
    SELECT id_pracownika_szefa_działu AS id_pracownika  
    FROM Dział  
    UNION  
    SELECT id_pracownika_kierownika_zespołu AS id_pracownika  
    FROM Zespół)  
USING (id_pracownika);
```

imię
Kasia
Olga
Wiktor

Klauzula INTERSECT

Klauzula INTERSECT

Klauzula INTERSECT łączy wyniki dwóch zapytań, zwracając te wiersze, które występują w obu wynikach

Klauzula INTERSECT eliminuje powtórzenia z wyniku końcowego

Klauzula INTERSECT

Przykład: Znaleźć numery tych pracowników, którzy są zarazem szefami działów i kierownikami zespołów

Klauzula INTERSECT

```
SELECT id_pracownika_szefa_działu AS id_pracownika  
FROM Dział  
INTERSECT  
SELECT id_pracownika_kierownika_zespołu  
FROM Zespół;
```

id_pracownika
1

Klauzuła EXCEPT

Klauzula EXCEPT

Klauzula EXCEPT łączy wyniki dwóch zapytań, zwracając te wiersze z wyników pierwszego zapytania, które nie występują w wynikach drugiego zapytania

Klauzula EXCEPT eliminuje powtórzenia z wyniku końcowego

Klauzula EXCEPT

Przykład: Znaleźć imiona tych pracowników, którzy są szefami działów i nie są kierownikami zespołów

Klauzula EXCEPT

Rozwiązanie 1:

```
SELECT imię  
FROM Pracownik p  
JOIN Dział d ON p.id_pracownika = d.id_pracownika_szefa_działu  
EXCEPT  
SELECT imię  
FROM Pracownik p  
JOIN Zespół z ON id_pracownika = z.id_pracownika_kierownika_zespołu;
```

imię
Kasia

Klauzula EXCEPT

Rozwiązanie 2:

```
SELECT imię  
FROM Pracownik JOIN (  
    SELECT id_pracownika_szefa_działu AS id_pracownika  
    FROM Dział  
    EXCEPT  
    SELECT id_pracownika_kierownika_zespołu AS id_pracownika  
    FROM Zespół)  
USING (id_pracownika);
```

imię
Kasia

Proszę o pytania :-)

Wspólne wyrażenia tabelowe,
czyli klauzula WITH

Wspólne wyrażenia tabelowe

CTE (Common Table Expression, czyli wspólne wyrażenie tabelowe) to nazwana, tymczasowa tabela, która istnieje tylko w czasie wykonywania jednego zapytania

CTE definiuje się klauzulą **WITH**

Wspólne wyrażenia tabelowe

Przykład 1: Znajdź tych pracowników, którzy w każdym zespole dostali wypłatę powyżej średniej wypłaty w tym zespole

Wspólne wyrażenia tabelowe

```
WITH ŚrednieWpłatyZespołów AS (  
    SELECT  
        id_zespołu,  
        AVG(kwota_wpłaty_PLN) AS średnia_wpłata  
    FROM PracownikZespołu  
    GROUP BY id_zespołu)  
SELECT  
    p.imię,  
    z.nazwa_zadania,  
    pz.kwota_wpłaty_PLN,  
    ROUND(swz.średnia_wpłata, 2) AS średnia_w_zespole  
FROM PracownikZespołu pz  
JOIN Pracownik p USING (id_pracownika)  
JOIN Zespół z USING (id_zespołu)  
JOIN ŚrednieWpłatyZespołów swz USING (id_zespołu)  
WHERE pz.kwota_wpłaty_PLN > swz.średnia_wpłata;
```

Wspólne wyrażenia tabelowe

imię	nazwa_zadania	kwota_wypłaty_PLN	średnia_w_zespole
Olga Lucyna	Projekt sterownika	15000.0	12500.0
	Projekt symulatora	20000.0	15666.67

Wspólne wyrażenia tabelowe

Przykład 2: Znajdź zespoły o sumarycznej kwocie wypłat wyższej niż średnia sumaryczna kwota wypłat zespołu

Wspólne wyrażenia tabelowe

```
WITH BudżetZespołu AS (  
    SELECT  
        id_zespołu,  
        SUM(kwota_wypłaty_PLN) AS suma_wypłat  
    FROM PracownikZespołu  
    GROUP BY id_zespołu),  
ŚredniBudżet AS (  
    SELECT  
        AVG(suma_wypłat) AS średni_budżet_zespołu  
    FROM BudżetZespołu)  
SELECT  
    z.nazwa_zadania,  
    bz.suma_wypłat AS budżet_zespołu,  
    sb.średni_budżet_zespołu AS średnia_wszystkich_zespołów,  
    p.imię AS imię_kierownika  
FROM BudżetZespołu bz JOIN ŚredniBudżet sb ON bz.suma_wypłat > sb.średni_budżet_zespołu  
JOIN Zespół z USING (id_zespołu)  
LEFT JOIN Pracownik p ON z.id_pracownika_kierownika_zespołu = p.id_pracownika;
```

Wspólne wyrażenia tabelowe

nazwa_zadania	budżet_zespołu	średnia_wszystkich_zespołów	imię_kierownika
Projekt symulatora	47000.0	36000.0	Ołga

Wspólne wyrażenia tabelowe

Przykład 3: Znajdź sumaryczne wypłaty pracowników poszczególnych działów

Wspólne wyrażenia tabelowe

```
WITH WypłatyPracowników AS (  
    SELECT  
        id_pracownika,  
        SUM(kwota_wypłaty_PLN) AS suma_wypłat_pracownika  
    FROM PracownikZespołu  
    GROUP BY id_pracownika)  
SELECT  
    d.nazwa AS nazwa_działu,  
    SUM(wp.suma_wypłat_pracownika) AS suma_wypłat  
FROM Dział d  
LEFT JOIN Pracownik p USING (id_działu)  
LEFT JOIN WypłatyPracowników wp USING (id_pracownika)  
GROUP BY d.nazwa;
```

Wspólne wyrażenia tabelowe

nazwa_działu	suma_wypłat
Dział Analiz	20000.0
Dział Wdrożeń	52000.0

Wspólne wyrażenia tabelowe

Przykład 4: Podaj, ile członków ma zespół każdego kierownika zespołu

Wspólne wyrażenia tabelowe

```
WITH PodsumowanieZespołu AS (  
    SELECT  
        id_zespołu,  
        COUNT(id_pracownika) AS liczba_członków  
    FROM PracownikZespołu  
    GROUP BY id_zespołu)  
SELECT p.imię, pz.liczba_członków  
FROM Zespół z  
JOIN Pracownik p ON z.id_pracownika_kierownika_zespołu =  
p.id_pracownika  
JOIN PodsumowanieZespołu pz USING (id_zespołu)  
ORDER BY pz.liczba_członków DESC;
```

Wspólne wyrażenia tabelowe

imię	liczba_członków
Olga	3
Wiktor	3
Wiktor	2

Wspólne wyrażenia tabelowe

Są też rekurencyjne CTE (klauszula **WITH RECURSIVE**), w których podzapytania odwołują się do samych siebie

Rekurencyjne CTE służą do przetwarzania drzew i grafów

Nie będziemy o nich mówić :-)

Proszę o pytania :-)

Podzapytania skorelowane

Podzapytania skorelowane

Podzapytanie skorelowane to takie podzapytanie, które odwołuje się do kolumn zapytania zewnętrznego

System zarządzania bazą danych wykonuje to podzapytanie osobno dla każdego wiersza rozpatrywanego przez zapytanie zewnętrzne

Podzapytania skorelowane

Przykład 1: Dla każdego zespołu znaleźć numer tego pracownika, który dostał najwyższą wypłatę w ramach tego zespołu

Podzapytania skorelowane

```
SELECT id_pracownika, id_zespołu, kwota_wypłaty_PLN
FROM PracownikZespołu pz1
WHERE kwota_wypłaty_PLN = (
    SELECT MAX(kwota_wypłaty_PLN)
    FROM PracownikZespołu pz2
    WHERE pz1.id_zespołu = pz2.id_zespołu);
```

id_pracownika	id_zespołu	kwota_wypłaty_PLN
1	1	15000.0
4	2	20000.0

Podzapytania skorelowane

Przykład 2: Dla każdego zespołu znaleźć imię tego pracownika, który dostał najwyższą wypłatę w ramach tego zespołu

Podzapytania skorelowane

```
SELECT id_zespołu, imię, kwota_wypłaty_PLN
FROM Pracownik JOIN PracownikZespołu pz1 USING (id_pracownika)
WHERE kwota_wypłaty_PLN = (
    SELECT MAX(kwota_wypłaty_PLN)
    FROM PracownikZespołu pz2
    WHERE pz1.id_zespołu = pz2.id_zespołu);
```

id_zespołu	imię	kwota_wypłaty_PLN
1	Olga	15000.0
2	Lucyna	20000.0

Podzapytania skorelowane

Przykład 3: Dla każdego zespołu znaleźć numery pracowników i kwoty ich wypłat, jeśli te wypłaty były niższe od średniej wypłaty w ramach tego zespołu

Podzapytania skorelowane

```
SELECT id_zespołu, id_pracownika, kwota_wypłaty_PLN
FROM PracownikZespołu pz1
WHERE kwota_wypłaty_PLN < (
    SELECT AVG(kwota_wypłaty_PLN)
    FROM PracownikZespołu pz2
    WHERE pz1.id_zespołu = pz2.id_zespołu);
```

id_zespołu	id_pracownika	kwota_wypłaty_PLN
1	2	10000.0
2	1	12000.0
2	2	15000.0

Podzapytania z warunkiem IN

Podzapytania z warunkiem IN

Zapytania zagnieżdżone z warunkiem **IN** zwracają te wiersze zapytania zewnętrznego, dla których wartość z danej kolumny znajduje się wśród wierszy zwróconych przez podzapytanie

Podzapytania z warunkiem IN

Przykład 1: Znaleźć nazwy tych zadań,
w ramach których były jakieś wypłaty

Podzapytania z warunkiem IN

```
SELECT nazwa_zadania  
FROM Zespół  
WHERE id_zespołu IN (  
    SELECT id_zespołu  
    FROM PracownikZespołu  
    WHERE kwota_wypłaty_PLN > 0);
```

nazwa_zadania
Projekt sterownika
Projekt symulatora

Podzapytania z warunkiem IN

Przykład 2: Znaleźć imiona tych pracowników, którzy dostali jakiegokolwiek wypłaty powyżej 15000 PLN

Podzapytania z warunkiem IN

```
SELECT imię  
FROM Pracownik  
WHERE id_pracownika IN (  
    SELECT id_pracownika  
    FROM PracownikZespołu  
    WHERE kwota_wypłaty_PLN > 15000);
```

imię
Lucyna

Podzapytania z warunkiem IN

Przykład 3: Znaleźć nazwy tych działów, których pracownicy dostali wypłaty w ramach zadania „projekt sterownika”

Podzapytania z warunkiem IN

Rozwiązanie z użyciem złączeń

```
SELECT DISTINCT d.nazwa  
FROM Dział d  
JOIN Pracownik p USING (id_działu)  
JOIN PracownikZespołu USING (id_pracownika)  
JOIN Zespół z USING (id_zespołu)  
WHERE z.nazwa_zadania = 'Projekt sterownika';
```

nazwa
Dział Wdrożeń

Podzapytania z warunkiem IN

Rozwiązanie z użyciem zapytań zagnieżdżonych

```
SELECT nazwa FROM Dział
WHERE id_działu IN (
    SELECT id_działu FROM Pracownik
    WHERE id_pracownika IN (
        SELECT id_pracownika FROM PracownikZespołu
        WHERE id_zespołu IN (
            SELECT id_zespołu FROM Zespół
            WHERE nazwa_zadania = 'Projekt sterownika'))));
```

nazwa
Dział Wdrożeń

Podzapytania z warunkami
EXISTS i NOT EXISTS

Podzapytania z warunkami EXISTS i NOT EXISTS

Zapytania zagnieżdżone z warunkami EXISTS i NOT EXISTS są używane z podzapytaniem skorelowanym

Operatory EXISTS i NOT EXISTS sprawdzają, czy podzapytanie zwraca choć jeden wiersz

Podzapytania z warunkami EXISTS i NOT EXISTS

Przykład 1: Wyszukać nazwy tych zadań,
w ramach których były jakieś wypłaty

Podzapytania z warunkami EXISTS i NOT EXISTS

```
SELECT nazwa_zadania
FROM Zespół z
WHERE EXISTS (
    SELECT *
    FROM PracownikZespołu pz
    WHERE kwota_wypłaty_PLN IS NOT NULL
    AND pz.id_zespołu = z.id_zespołu);
```

nazwa_zadania
Projekt sterownika
Projekt symulatora

Podzapytania z warunkami EXISTS i NOT EXISTS

Przykład 2: Wyszukać imiona tych pracowników, którzy nie mieli żadnych wypłat

Podzapytania z warunkami EXISTS i NOT EXISTS

```
SELECT imię  
FROM Pracownik p  
WHERE NOT EXISTS (  
    SELECT *  
    FROM PracownikZespołu pz  
    WHERE kwota_wypłaty_PLN IS NOT NULL  
    AND pz.id_pracownika = p.id_pracownika);
```

imię
Kasia

Podzapytania z warunkami EXISTS i NOT EXISTS

Przykład 3: Wyszukać imiona tych pracowników, którzy braли udział w każdym zadaniu

Podzapytania z warunkami EXISTS i NOT EXISTS

Rozwiązanie 1: Wyszukać imiona tych pracowników, dla których nie istnieje takie zadanie, w którym nie brali udziału

```
SELECT imię
FROM Pracownik p
WHERE NOT EXISTS (
    SELECT *
    FROM Zespół z
    WHERE NOT EXISTS (
        SELECT *
        FROM PracownikZespołu pz
        WHERE pz.id_pracownika = p.id_pracownika
        AND pz.id_zespołu = z.id_zespołu));
```

imię
Olga
Wiktor

Podzapytania z warunkami EXISTS i NOT EXISTS

Rozwiązanie 2

```
SELECT imię
FROM Pracownik p JOIN (
    SELECT id_pracownika
    FROM PracownikZespołu
    GROUP BY id_pracownika
    HAVING COUNT(id_zespołu) = (
        SELECT COUNT(DISTINCT id_zespołu)
        FROM PracownikZespołu))
    USING (id_pracownika);
```

imię
Olga
Wiktor

Podzapytania w kłauzułach
FROM i SELECT

Podzapytania w klauzulach FROM i SELECT

```
SELECT ...  
FROM (  
    SELECT ...);
```

Podzapytania w kłauzułach FROM i SELECT

Przykład: Znajdź największą sumę wypłat
wśród wszystkich pracowników

Podzapytania w klauzulach FROM i SELECT

```
SELECT id_pracownika, MAX(suma)
FROM (
    SELECT id_pracownika, SUM(kwota_wypłaty_PLN) AS suma
    FROM PracownikZespołu
    GROUP BY id_pracownika);
```

id_pracownika	MAX(suma)
1	27000.0

Podzapytania w klauzulach FROM i SELECT

```
SELECT ..., (SELECT ...)  
FROM ...;
```

Podzapytania w klauzulach FROM i SELECT

Przykład: Dla każdego pracownika znajdź sumę kwot jego wypłat

Podzapytania w kłauzułach FROM i SELECT

Rozwiązanie 1 z użyciem podzapytania

```
SELECT id_pracownika, imię, (  
    SELECT SUM(kwota_wypłaty_PLN) AS suma  
    FROM PracownikZespołu pz  
    WHERE pz.id_pracownika = p.id_pracownika)  
FROM Pracownik p;
```

id_pracownika	imię	(
1	Olga	27000.0
2	Wiktor	25000.0
3	Kasia	NULL
4	Lucyna	20000.0

Podzapytania w kłauzułach FROM i SELECT

Rozwiązanie 2 z użyciem złączenia

```
SELECT id_pracownika, imię, SUM(kwota_wypłaty_PLN) AS suma  
FROM Pracownik p JOIN PracownikZespołu pz USING (id_pracownika)  
GROUP BY id_pracownika, imię;
```

id_pracownika	imię	(
1	Olga	27000.0
2	Wiktor	25000.0
3	Kasia	NULL
4	Lucyna	20000.0

To już koniec :-)

Tematy dzisiejszego wykładu

Z tego wykładu dowiedziecie się:

- + o klauzulach UNION i UNION ALL
- + o klauzuli INTERSECT
- + o klauzuli EXCEPT
- + o wspólnych wyrażeniach tabel owych (Common Table Expressions, CTE), czyli o klauzuli WITH
- + o innych rodzajach zapytań zagnieżdżonych

Proszę o pytania :-)

Zapraszam was na następny wykład. Dowiedziecie się z niego:

- + o indeksach (INDEX)
- + o widokach (VIEW)
- + o transakcjach (TRANSACTION)
- + o funkcji ROW_NUMBER

Podziękowania

Podziękowania @→+—

Dziękuję:

+ prof. Stanisławowi Koziełskiemu

Źródła grafik

+ <https://gemini.google.com/>